

Fundamentals of Web Applications

Frontend Development: Grid and Responsive Layout in CSS

Lecture 17 and 18 – May 7 and 13, 2026

Michael Soprano - michael.soprano@uniud.it

University of Udine - Department of Mathematics,
Computer Science, and Physics (DMIF)

Outline

1. Two-Dimensional Layouts
2. Responsive Layout
3. Positioning and Layers

Layout Context Recap

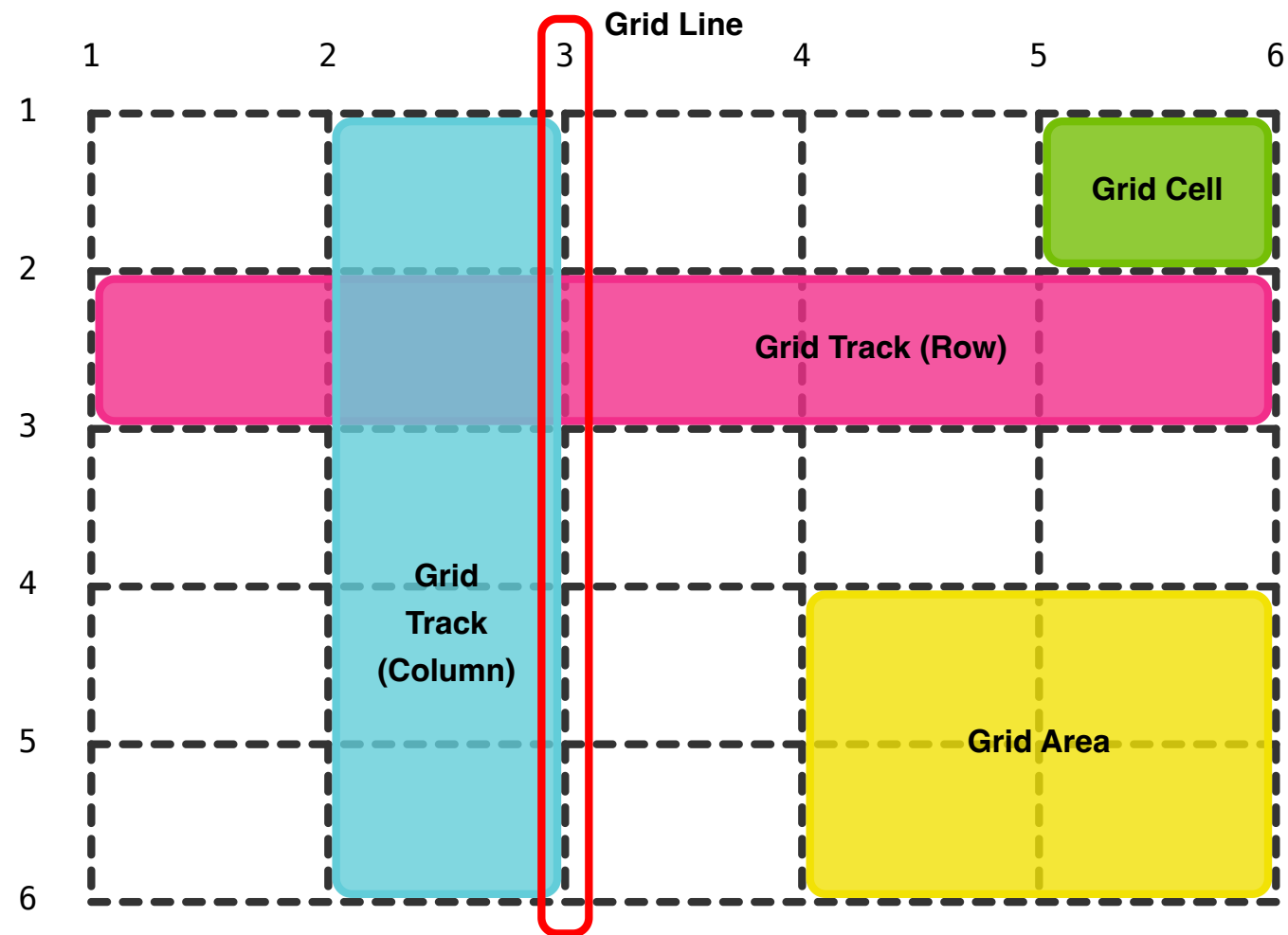
- CSS layout usually starts from the **normal flow**
- Flexbox and Grid are applied to a **container**
- Only the container's **direct children** become layout items
- Flexbox is strongest for **one-dimensional** arrangements
- Grid is designed for layouts that need **rows and columns** together

Grid Rows and Columns

- Grid arranges items inside a two-dimensional **structure**
- The container defines **columns, rows, tracks,** and **gaps**
- The direct children become **grid items**
- Grid items can occupy one cell or span multiple cells
- Container properties define the structure, while item properties define **placement**

Role	Examples
Container	<code>grid-template-columns</code> , <code>grid-template-rows</code> , <code>grid-template-areas</code> , <code>gap</code>
Item	<code>grid-column</code> , <code>grid-row</code> , <code>grid-area</code>

Grid Structure Overview



Main parts of a grid layout structure

Column Tracks: `grid-template-columns`

- `grid-template-columns` defines the **column structure**
- The `fr` unit distributes available space between **flexible columns**
- Equal `fr` values create **equal-width columns**
- Additional rows can be created automatically when more items are added

```
.gallery {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 1rem;  
}
```

Adaptive Columns: `auto-fit` and `minmax()`

- A fixed number of columns can become too narrow on **small screens**
- `auto-fit` creates as many columns as fit
- `minmax(220px, 1fr)` keeps each column readable before it grows
- The grid adapts without changing the **HTML structure**

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 1rem;  
}
```

Automatic Placement

- Grid items do not always need explicit **coordinates**
- By default, the browser places items into the next available **grid cell**
- Additional rows are created automatically when more items are added
- Repeated content such as **galleries** and **card lists** can follow this automatic flow

Example: Source Code

- Sample **resource**: AU_grid-gallery/index.html
- The gallery contains several cards with the same **HTML structure**

```
←!— Head omitted for brevity →  
<body>  
  <main class="page">  
    <h1>Course Topics</h1>  
  
    <section class="gallery">  
      <article class="card">  
        <h2>HTML Structure</h2>  
        <p>Semantic elements define the meaning of the content.</p>  
      </article>  
      <article class="card">  
        <h2>CSS Layout</h2>  
        <p>Layout rules arrange boxes into readable interfaces.</p>  
      </article>  
      ←!— Additional cards omitted for brevity →  
    </section>  
  </main>  
</body>
```

Example: Source Code

- Sample **resource**: [AU_grid-gallery/style.css](#)
- `auto-fit` and `minmax()` create **adaptive columns** for the gallery
- The browser places the cards automatically into the available **grid cells**

```
body { margin: 0; font-family: Arial, Helvetica, sans-serif; line-height: 1.5; }

.page {
  max-width: 960px;
  margin: 2rem auto;
  padding: 0 1rem;
}

.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.card { padding: 1rem; border: 1px solid #cccccc; background-color: #eaf3ff; }
```

Example: Result

- The cards are placed automatically into **grid cells**
- The browser decides how many columns fit in the available **container width**
- New rows are created automatically when there are more cards
- The HTML stays simple while Grid controls the **visual structure**

HTML Structure

Semantic elements define the meaning of the content.

CSS Layout

Layout rules arrange boxes into readable interfaces.

JavaScript Basics

Scripts add behavior and simple interactivity.

Forms

Controls collect input and send structured data.

Automatic and Explicit Placement

- Grid can place repeated items automatically into the available **cells**
- Some items can also receive explicit **placement rules**
- Explicit placement gives an item a different **span** or visual region
- The same grid can combine automatic flow and selected manual placement

Item Spans: `grid-column` and `grid-row`

- A grid item can span across multiple **columns** or **rows**
- Line numbers identify where the item **starts** and where it **ends**
- `grid-column` controls the horizontal span
- `grid-row` controls the vertical span

```
.feature { grid-column: 1 / 3; }
```

```
.sidebar { grid-row: 2 / 4; }
```

Named Layout Areas

- Line numbers are precise, but they can become hard to read in larger layouts
- Some layouts read more clearly as named **areas**
- `grid-template-areas` describes the layout as a small **textual map**
- The map is written on the **grid container**

```
.page-layout {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "main sidebar";  
  grid-template-columns: 2fr 1fr;  
}
```

Reading Area Maps

- Each quoted line represents one **grid row**
- Each name inside a line represents one **grid cell**
- Repeated names create a larger **layout area**
- `grid-template-columns` gives sizes to the mapped **columns**

	Column 1	Column 2
Row 1	header	header
Row 2	main	sidebar

```
grid-template-areas:  
  "header header"  
  "main sidebar";
```

Area Spans and Empty Cells

- A named area spans exactly the cells where its **name appears**
- All cells with the same name must form one **contiguous rectangle**
- Use `.` when a cell should be intentionally **empty**
- The map should show the full intended **visual structure**

```
grid-template-areas:  
  "header header header"  
  "main main sidebar";
```

/ With an empty cell */*

```
grid-template-areas:  
  "header header ."  
  "main sidebar .";
```


Assigning Items to Areas

- The area map defines the available **layout areas**
- Each grid item is assigned to **one named area** with `grid-area`
- The item occupies all cells where that **area name** appears
- Changing the map can rearrange the layout without changing the **HTML structure**

```
.page-layout {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "main sidebar";  
  grid-template-columns: 2fr 1fr;  
  gap: 1rem;  
}  
  
.header { grid-area: header; }  
.main { grid-area: main; }  
.sidebar { grid-area: sidebar; }
```

Semantic Order and Layout Areas

- Semantic elements describe the **meaning** of the content
- Grid area names describe the **visual regions** used by CSS
- Similar names make the relationship between semantics and layout easier to read
- Visual placement does not change the underlying **source order**
- Use Grid to improve layout, not to repair an incorrect **document structure**

```
<header class="header">...</header>  
<section class="main">...</section>  
<aside class="sidebar">...</aside>
```

```
.header { grid-area: header; }  
.main { grid-area: main; }  
.sidebar { grid-area: sidebar; }
```

Example: Source Code

- Sample **resource**: AV_grid-page-regions/index.html
- The document uses semantic elements for the main **content regions**
- CSS will assign these elements to named **layout areas**

```
←!— Head omitted for brevity —→  
<body>  
  <main class="page-layout">  
    <header class="header">  
      <h1>Course Dashboard</h1>  
    </header>  
  
    <section class="main">  
      <h2>Current Topic</h2>  
      <p>Modern CSS layout combines structure, spacing, and responsive behavior.</p>  
    </section>  
  
    <aside class="sidebar">  
      <h2>Resources</h2>  
      <ul>  
        <li>Slides</li>  
        <li>Examples</li>  
        <li>Exercises</li>  
      </ul>  
    </aside>  
  </main>  
</body>
```

Example: Source Code

- Sample **resource**: [AV_grid-page-regions/style.css](#)
- The container defines the **visual map** of the layout
- Each direct child is assigned to one named **grid area**

```
.page-layout {  
  max-width: 900px;  
  margin: 2rem auto;  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "main sidebar";  
  grid-template-columns: 2fr 1fr;  
  gap: 1rem;  
}  
  
.header { grid-area: header; }  
.main { grid-area: main; }  
.sidebar { grid-area: sidebar; }
```

Example: Result

- The header spans both **columns**
- The main content and sidebar occupy separate **grid areas**
- The HTML describes the content structure
- CSS defines the two-dimensional **visual arrangement**

Course Dashboard

Current Topic

Modern CSS layout combines structure, spacing, and responsive behavior.

Resources

- Slides
- Examples
- Exercises

Flexbox and Grid

- Flexbox arranges items mainly along one **main axis**
- Grid controls **rows** and **columns** together
- Both systems are applied to a **container**
- The choice depends on the **structure** the layout needs

Need	Better Fit
Navigation links in one row	Flexbox
Toolbar regions	Flexbox
Card gallery	Grid
Page regions with rows and columns	Grid

Available Space Changes

- The same component may be rendered in very different **available spaces**
- A layout can be readable at one width and cramped at another
- The HTML structure should not need to change for each screen size
- CSS can adapt the visual arrangement when space changes

Responsive Layout

- A **responsive layout** adapts its visual arrangement to the available space
- The goal is to keep content **readable, usable**, and visually stable
- A different arrangement is introduced only when it improves the component
- Responsive design is therefore based on **layout needs**, not device names

Viewport and Containers

- The **viewport** is the visible area of the browser window
- A component is also rendered inside a specific **container**
- Available space can depend on the viewport, the container, or both
- Responsive layout considers both the page and the **component**

Responsive Breakpoints

- A **breakpoint** is the condition where CSS changes the layout
- It marks the point where the current arrangement no longer works well
- Different components may need different **breakpoints**
- In CSS, breakpoints are commonly implemented with **media queries**

Media Queries: @media

- A media query applies CSS only when a **condition** is true
- The rules inside @media are used only when the query matches
- The condition can test **width, height, orientation**, and other media features
- Width conditions are the most common starting point for responsive layout

```
@media (condition) {  
  selector {  
    property: value;  
  }  
}
```

Width-Based Conditions

- `min-width` applies rules from a given width **upward**
- `max-width` applies rules up to a given **maximum width**
- A range can target widths between two values
- A mobile-first layout commonly uses `min-width`

```
@media (min-width: 720px) {  
  /* Rules for viewports at least 720px wide */  
}  
  
@media (max-width: 719px) {  
  /* Rules for viewports up to 719px wide */  
}  
  
@media (min-width: 720px) and (max-width: 1024px) {  
  /* Rules for an intermediate width range */  
}
```

Mobile-First Strategy

- In a mobile-first strategy, the base CSS describes the **narrow layout**
- Wider layouts are added progressively with `min-width` queries
- This keeps the first layout simple
- It also avoids overriding many desktop-oriented rules

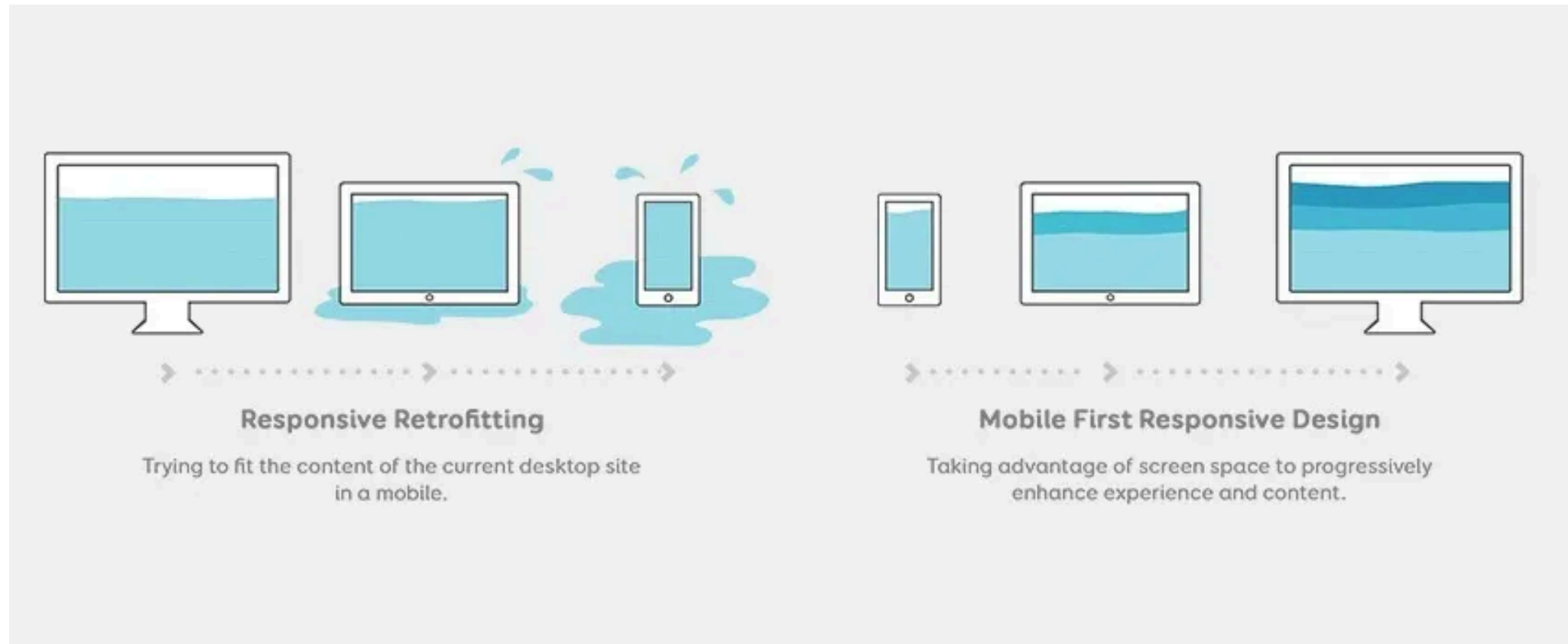
```
.feature-card {  
  display: grid;  
  gap: 1rem;  
}  
  
@media (min-width: 720px) {  
  .feature-card {  
    grid-template-columns: 180px 1fr;  
  }  
}
```

Bootstrap Grid Pattern

- Many CSS frameworks adopt a **mobile-first** responsive strategy
- In Bootstrap, the smallest layout is defined first
- Breakpoint-specific classes add wider layouts progressively
- The same idea appears in custom CSS with `min-width` media queries

```
<div class="row">
  <article class="col-12 col-md-6 col-lg-4">Card</article>
  <article class="col-12 col-md-6 col-lg-4">Card</article>
  <article class="col-12 col-md-6 col-lg-4">Card</article>
</div>
```

Mobile-First Design



Mobile-first design starts from narrow screens and progressively enhances the layout

Example: Source Code

- Sample **resource**: [AW_responsive-media-card/index.html](https://www.audible.com.au/responsive-media-card/index.html)
- The card contains an image, text content, and an action link

```
←!— Head omitted for brevity —→  
<body>  
  <main class="page">  
    <article class="feature-card">  
        
  
      <div class="feature-content">  
        <p class="label">Frontend Development</p>  
        <h1>Responsive Components</h1>  
        <p>  
          A component can change its visual layout while keeping  
          the same HTML structure.  
        </p>  
        <a href="#">View materials</a>  
      </div>  
    </article>  
  </main>  
</body>
```

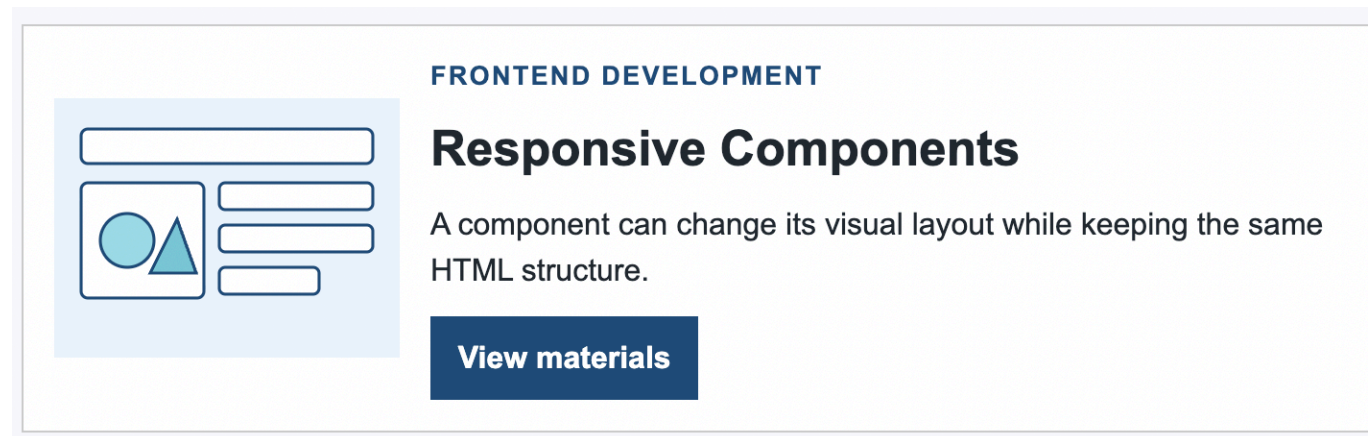
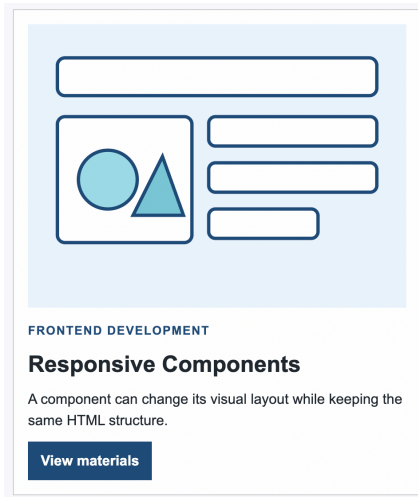

Example: Mobile-First CSS

- Sample **resource**: [AW_responsive-media-card/style.css](#)
- The base rules create a **narrow layout**
- The media query adds a **wider layout** from **720px** upward

```
.feature-card {  
  /* Base layout: one-column grid for narrow viewports */  
  display: grid;  
  gap: 1rem;  
}  
  
@media (min-width: 720px) {  
  .feature-card {  
    /* Wider layout: fixed media column and flexible text column */  
    grid-template-columns: 180px 1fr;  
  }  
}
```

Example: Result

- Below 720px, the card remains a **vertical layout**
- From 720px upward, the card becomes a **two-column layout**
- The image occupies the first column
- The text content occupies the flexible second column



The same card changes from a vertical layout to a two-column layout

Responsive Media

- Responsive layout is not only about arranging **boxes**
- Images and videos also occupy space inside **components**
- Their intrinsic size can be larger than the available **container space**
- Responsive media rules keep media inside flexible layouts

```
img,  
video {  
  max-width: 100%;  
  height: auto;  
}
```

Fluid Images

- A fluid image scales with its **container**
- `max-width: 100%` prevents the image from becoming wider than the container
- `height: auto` preserves the original **aspect ratio**
- The full image remains visible while it scales down

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Stable Media Frames: `aspect-ratio` and `object-fit`

- Some components need media with a consistent **visual shape**
- `aspect-ratio` reserves predictable **layout space**
- `object-fit: cover` fills the frame and may crop the content
- `object-fit: contain` shows the full content and may leave empty space

```
.card img {  
  width: 100%;  
  aspect-ratio: 16 / 9;  
  object-fit: cover;  
}
```

Choosing the Right Media Rule

- Use **fluid media** when the full content should remain visible
- Use a **stable frame** when the component needs a consistent shape
- Use `object-fit: contain` when cropping would remove important content
- Use `object-fit: cover` when visual consistency matters more than full visibility

Example: Source Code

- Sample **resource**: [AX_responsive-media-rules/index.html](https://ax-responsive-media-rules/index.html)
- The example compares a **fluid image** and a **framed image**
- Both images use the same source file, but different CSS rules

```
←!— Head omitted for brevity —→
<body>
  <main class="page">
    <h1>Responsive Media Rules</h1>

    <section class="media-grid">
      <article class="media-card">
        <h2>Fluid Image</h2>
        
        <p>The full image remains visible and scales with the container.</p>
      </article>

      <article class="media-card">
        <h2>Framed Image</h2>
        
        <p>The image fills a stable frame and may be cropped.</p>
      </article>
    </section>
  </main>
</body>
```

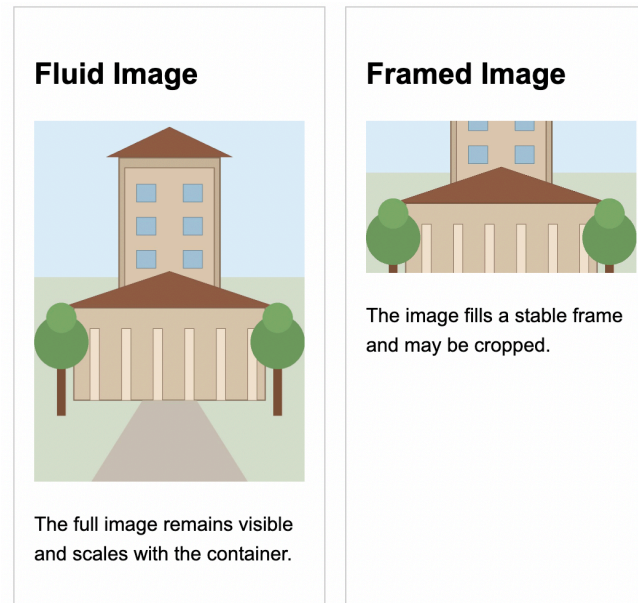
Example: Source Code

- Sample **resource**: [AX_responsive-media-rules/style.css](#)
- `.fluid-image` prioritizes **full visibility**
- `.framed-image` prioritizes a stable **component shape**

```
.fluid-image {  
  max-width: 100%;  
  height: auto;  
}  
  
.framed-image {  
  width: 100%;  
  aspect-ratio: 16 / 9;  
  object-fit: cover;  
}
```


Example: Result

- The **fluid image** keeps the full picture visible
- The **framed image** keeps a consistent rectangular shape
- `object-fit: cover` fills the frame by cropping if needed
- The right rule depends on the role of the media in the component



Fluid media preserves the full image, while framed media preserves the component shape

Responsive Layout Checklist

- Start from a readable **small-screen structure**
- Use Flexbox or Grid to arrange related **components**
- Add breakpoints when the layout actually needs to change
- Keep media items inside their **containers**
- Choose fluid media or framed media based on the media **role**

Special Placement Problems

- Flexbox and Grid arrange the main **layout structure**
- Some interface elements need more specific **placement**
- A badge may sit in the corner of a card
- A button may stay fixed near the edge of the viewport
- A label may remain visible while the page scrolls

Positioning Mode: `position`

- `static` is the default mode, and the element follows the **normal flow**
- Other values create **positioned elements**
- Positioned elements can use `top`, `right`, `bottom`, and `left`
- The meaning of those offsets depends on the selected **positioning mode**
- Positioning should not replace Flexbox or Grid for ordinary layout structure

```
.badge {  
  position: absolute;  
  top: 1rem;  
  right: 1rem;  
}
```

Local Reference: `relative`

- `position: relative` keeps the element in the **normal flow**
- Offsets move the element visually from its **normal-flow position**
- The original space is still preserved
- It often creates a local **positioning reference** for absolutely positioned children

```
.card {  
  position: relative;  
}
```

Local Overlay: `absolute`

- `position: absolute` removes the element from the **normal flow**
- The element is positioned relative to the nearest **positioned ancestor**
- Offsets place the element from the edges of that **positioning reference**
- This pattern places badges, icons, tooltips, and small overlays inside a component

```
.card {  
  position: relative;  
}  
  
.badge {  
  position: absolute;  
  top: 1rem;  
  right: 1rem;  
}
```

Viewport and Scroll Placement

- `position: fixed` anchors an element to the **viewport**
- The element stays in the same visual position while the document scrolls
- `position: sticky` starts in the **normal flow**
- A sticky element becomes fixed only after a scroll **threshold** is reached
- Both modes should be checked carefully because they can cover **content**

```
.help-button {  
  position: fixed;  
  right: 1rem;  
  bottom: 1rem;  
}  
  
.section-label {  
  position: sticky;  
  top: 0;  
}
```

Layer Order: `z-index`

- Positioned elements can overlap other elements
- `z-index` controls the **stacking order**
- Higher values appear in front of lower values
- Use it only when overlap is intentional

```
.card {  
  position: relative;  
}  
  
.badge {  
  position: absolute;  
  top: 1rem;  
  right: 1rem;  
  z-index: 1;  
}
```


Example: Source Code

- Sample **resource**: AY_positioned-badge-card/index.html
- The card list uses Flexbox for the main **layout**
- The badge is positioned inside one card as a local **overlay**

```
←!— Head and body omitted for brevity —→  
<main class="page">  
  <section class="card-list">  
    <article class="card">  
      <span class="badge">New</span>  
      <h1>Modern Layout</h1>  
      <p>Flexbox arranges the card list, while positioning places the badge.</p>  
      <a href="#">Open notes</a>  
    </article>  
  
    <article class="card">  
      <h1>Responsive Media</h1>  
      <p>Other cards follow the same flex layout.</p>  
      <a href="#">View example</a>  
    </article>  
  </section>  
</main>
```

Example: Flex Layout

- Sample **resource**: [AY_positioned-badge-card/style.css](#)
- `.card-list` creates the **Flexbox layout**
- `.card` becomes both a flex item and a local **positioning reference**

```
.card-list {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}  
  
.card {  
  position: relative;  
  flex: 1 1 18rem;  
  padding: 1.5rem;  
  border: 1px solid #cccccc;  
}
```

Example: Positioned Badge

- Sample **resource**: [AY_positioned-badge-card/style.css](#)
- `.badge` is positioned relative to the nearest **positioned ancestor**
- In this example, that ancestor is `.card`
- `top` and `right` place the badge from the card's **top-right corner**

```
.badge {  
  position: absolute;  
  top: 1rem;  
  right: 1rem;  
  padding: 0.25rem 0.6rem;  
  background-color: #1f4e79;  
  color: #ffffff;  
  font-weight: 700;  
  z-index: 1;  
}
```

Example: Result

- Flexbox arranges the cards as **layout items**
- The badge appears as a small **overlay** inside one card
- Positioning affects the badge, not the whole card list
- The main layout remains controlled by **Flexbox**

Modern Layout

New

Flexbox arranges this card in a row, while positioning places the badge inside the card.

[Open notes](#)

Responsive Media

Other cards follow the same flex layout without needing a positioned badge.

[View example](#)

Reading the Example

- The card list remains a normal **Flexbox layout**
- Each card keeps its own space inside that layout
- `position: relative` gives the card a local **reference**
- `position: absolute` lets the badge use that reference for local **placement**
- `top`, `right`, and `z-index` affect only the badge's **overlay behavior**

Positioning Summary

- Use Flexbox or Grid for the main **layout structure**
- Use `relative` to create a local **positioning reference**
- Use `absolute` for local overlays inside a component
- Use `fixed` and `sticky` for viewport or scroll-aware elements
- Use `z-index` only when elements intentionally **overlap**

Layout Decisions

- Start from meaningful **HTML structure**
- Let normal flow handle simple arrangements
- Use Flexbox for mainly **one-dimensional** groups
- Use Grid for structures based on **rows and columns**
- Use responsive rules when available **space** changes the layout
- Use positioning only for special **placement**, overlays, and layers

Looking Ahead: Selectors

- Layout rules depend on selecting the right **elements**
- Simple selectors cover many basic rules
- More complex interfaces need more precise selection patterns
- Advanced selectors target states, combinations, and repeated structures
- Better selectors make stylesheets more expressive and easier to maintain

Bibliography

- Duckett, J. (2011). *HTML and CSS: Design and Build Websites*. John Wiley & Sons
 - Chapter 15: Layout
 - Chapter 17: HTML5 Layout
- MDN Web Docs. *Basic concepts of grid layout*
 - https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Basic_concepts_of_grid_layout
- MDN Web Docs. *Grid template areas*
 - https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Grid_template_areas

Bibliography (cont.)

- MDN Web Docs. *Media queries*
 - https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries
- MDN Web Docs. *Responsive design*
 - https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
- Bootstrap Documentation. *Grid system*
 - <https://getbootstrap.com/docs/5.3/layout/grid/>

Bibliography (cont.)

- MDN Web Docs. *position*
 - <https://developer.mozilla.org/en-US/docs/Web/CSS/position>
- MDN Web Docs. *z-index*
 - <https://developer.mozilla.org/en-US/docs/Web/CSS/z-index>
- MDN Web Docs. *aspect-ratio*
 - <https://developer.mozilla.org/en-US/docs/Web/CSS/aspect-ratio>
- MDN Web Docs. *object-fit*
 - <https://developer.mozilla.org/en-US/docs/Web/CSS/object-fit>